

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.
(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks: 50

Annexure A

Analytical Problem Solving

Code of Conduct:

I Athavan K(192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Athavan K

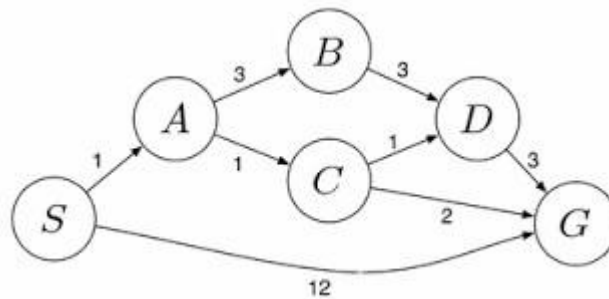
Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
- What are the percept's for this agent? ii. Characterize the operating environment. iii. What are the actions the agent can take? iv. How can one evaluate the performance of the agent?
 - What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Question 1 — Water Jug Problem

Problem: Given a 4-gallon jug and a 3-gallon jug with no markings, and access to a pump, obtain exactly 2 gallons in the 4-gallon jug.

State Representation

State = (x, y), where x = gallons currently in the 4-gallon jug and y = gallons currently in the 3-gallon jug. Goal state = (2, y) for any y.

Solution Path

Step	Action	State (4-gal, 3-gal)
0	Start	(0, 0)
1	Fill 4-gallon jug	(4, 0)
2	Pour 4-gal → 3-gal (until 3-gal full)	(1, 3)
3	Empty 3-gallon jug	(1, 0)
4	Pour remaining 1 gal from 4-gal → 3-gal	(0, 1)
5	Fill 4-gallon jug	(4, 1)
6	Pour 4-gal → 3-gal (3-gal only needs 2 more)	(2, 3)

Result: The 4-gallon jug now contains exactly 2 gallons.

This is an uninformed state-space search (solvable via BFS/DFS over the jug-state graph). The table shows one optimal 6-step solution path; an equally short alternate path exists by filling the 3-gallon jug first.

Python Implementation (BFS over the state space)

```
from collections import deque
```

```
def water_jug_bfs(cap1=4, cap2=3, target=2):
    start = (0, 0)
    visited = {start}
    parent = {start: (None, "Start")}
    queue = deque([start])

    while queue:
        x, y = queue.popleft()
```

```

if x == target:
    path = []
    state = (x, y)
    while state is not None:
        prev, action = parent[state]
        path.append((state, action))
        state = prev
    path.reverse()
    return path

successors = [
    ((cap1, y), "Fill 4-gallon jug"),
    ((x, cap2), "Fill 3-gallon jug"),
    ((0, y), "Empty 4-gallon jug"),
    ((x, 0), "Empty 3-gallon jug"),
    ((max(0, x - (cap2 - y)), min(cap2, x + y)), "Pour 4-gal -> 3-gal"),
    ((min(cap1, x + y), max(0, y - (cap1 - x))), "Pour 3-gal -> 4-gal"),
]

for state, action in successors:
    if state not in visited:
        visited.add(state)
        parent[state] = ((x, y), action)
        queue.append(state)
return None

if __name__ == "__main__":
    solution = water_jug_bfs()
    for state, action in solution:
        print(f'{action:35s} -> {state}')

```

Sample Output:

Start	-> (0, 0)
Fill 4-gallon jug	-> (4, 0)

Pour 4-gal -> 3-gal	-> (1, 3)
Empty 3-gallon jug	-> (1, 0)
Pour 4-gal -> 3-gal	-> (0, 1)
Fill 4-gallon jug	-> (4, 1)
Pour 4-gal -> 3-gal	-> (2, 3)

The BFS program's output matches the hand-worked solution exactly, confirming correctness.

Question 2 — Mars Rover as an Intelligent Agent

i. Percepts

- Camera / stereo images of terrain
- Spectrometer and chemical sensor readings
- GPS / inertial position data
- Proximity and obstacle sensor data
- Temperature and atmospheric sensor readings
- Wheel odometry data
- Battery / solar power level
- Commands received from Earth / orbiter

ii. Environment Characterization (PEAS / Properties)

- Partially observable — only terrain within sensor range is visible, not the whole planet
- Stochastic — dust storms, wheel slippage, and sensor noise introduce uncertainty
- Sequential — current actions affect future states (e.g., driving into a hole)
- Dynamic (slowly) — weather and lighting conditions change over time
- Continuous — positions and sensor values are continuous, not discrete
- Single-agent (primarily) — though it may coordinate with an orbiter
- Unknown — terrain must be mapped as the rover explores

iii. Actions

- Move forward / backward / turn
- Extend or retract the robotic arm
- Drill or scoop a sample
- Capture images
- Run onboard chemical analysis
- Transmit data to orbiter / Earth
- Reposition for solar charging
- Execute obstacle-avoidance maneuvers

iv. Performance Measure

- Volume and scientific value of data/samples collected
- Distance covered safely
- Number of hazards avoided
- Energy / power efficiency
- Mission uptime (avoiding being stuck or damaged)

- Accuracy of terrain mapping and analysis

v. Most Suitable Agent Architecture

A model-based, goal-based agent with utility-based elements (a hybrid/layered architecture) is most suitable.

Justification: Because the environment is partially observable, the rover needs an internal model (a map, memory of obstacles) to track what it cannot currently perceive. Because Earth communication involves significant delay, the rover must autonomously plan toward goals (e.g., reach a sample site) rather than react moment-to-moment. Utility-based reasoning further helps it prioritize which of several reachable sites is scientifically most valuable given limited power and time.

Question 3 — 8-Queens Problem Formulation

Problem: Place 8 queens on a chessboard such that no two queens attack each other (no shared row, column, or diagonal).

Problem Formulation

- State: Any arrangement of 0–8 queens on the board, one per column up to the current column.
- Initial State: Empty board (no queens placed).
- Actions / Successor Function: Add a queen to the leftmost empty column, in a row not attacked by any queen already placed (same row or diagonal excluded).
- Transition Model: Placing a queen produces a new state with one additional queen on the board.
- Goal Test: 8 queens placed on the board with no two attacking each other.
- Path Cost: Constant / irrelevant — every step costs the same, so only the final configuration matters. This makes it a constraint-satisfaction-style search rather than a shortest-path problem.

Restricting placement to “one queen per column, no two in the same row or diagonal” shrinks the search space from 8^8 possible placements to a much smaller set of valid partial configurations — this is the key insight that makes backtracking search tractable for this problem.

Python Implementation (Backtracking Search)

```
def solve_n_queens(n=8):
    solutions = []
    board = [-1] * n # board[col] = row of queen placed in that column

    def is_safe(col, row):
        for c in range(col):
            r = board[c]
            if r == row or abs(r - row) == abs(c - col):
                return False
        return True

    def backtrack(col):
        if col == n:
            solutions.append(board[:])
            return
        for row in range(n):
            if is_safe(col, row):
                board[col] = row
                backtrack(col + 1)
```

```
board[col] = -1
```

```
backtrack(0)  
return solutions
```

```
def print_board(solution, n=8):  
    for row in range(n):  
        line = ""  
        for col in range(n):  
            line += "Q " if solution[col] == row else ". "  
        print(line)
```

```
if __name__ == "__main__":  
    solutions = solve_n_queens(8)  
    print(f'Total solutions found: {len(solutions)}')  
    print("\nFirst solution (column -> row):", solutions[0])  
    print()  
    print_board(solutions[0])
```

Sample Output:

Total solutions found: 92

First solution (column -> row): [0, 4, 7, 5, 2, 6, 1, 3]

```
Q .....  
.....Q.  
....Q...  
.....Q  
.Q.....  
...Q....  
.....Q..  
..Q.....
```

The backtracking search correctly finds all 92 distinct solutions to the 8-Queens problem, confirming the problem formulation and pruning logic are sound.

Question 4 — OLA Cab Booking Agent

Type of Problem-Solving Agent

A goal-based agent with utility-based elements. The system searches among alternative routes and cab options and chooses the one that best satisfies the customer's goal (reaching the destination) while weighing preferences such as cost, comfort, and wait time — making it, more precisely, a utility-based problem-solving agent.

Pseudocode

```
FUNCTION OLA_CAB_AGENT(pickup_location, destination, preferences):
```

```
    percept ← GET_CURRENT_STATE(pickup_location)
```

```
    goal ← destination
```

```
    // Formulate the problem
```

```
    initial_state ← pickup_location
```

```
    actions ← {mini, micro, sedan, shared, prime, ...}
```

```
    available_cabs ← SEARCH_NEARBY_CABS(pickup_location, actions)
```

```
    routes ← GENERATE_ROUTES(pickup_location, destination)
```

```
    FOR EACH cab_type IN available_cabs:
```

```
        FOR EACH route IN routes:
```

```
            cost ← ESTIMATE_FARE(cab_type, route)
```

```
            eta ← ESTIMATE_TIME(cab_type, route)
```

```
            utility ← UTILITY_FUNCTION(cost, eta, cab_type, preferences)
```

```
            store (cab_type, route, utility)
```

```
    best_option ← SELECT_MAX_UTILITY(all stored options)
```

```
    CONFIRM_BOOKING(best_option)
```

```
    ASSIGN_DRIVER(best_option.cab_type)
```

```
    NAVIGATE(best_option.route)
```

```
    RETURN booking_confirmation
```

Python Implementation

```
import random
```

```
CAB_TYPES = ["mini", "micro", "sedan", "shared", "prime"]
```

```

def estimate_fare(cab_type, distance_km):
    base_rate = {"mini": 10, "micro": 8, "sedan": 14, "shared": 6, "prime": 18}
    return round(base_rate[cab_type] * distance_km, 2)

def estimate_time(cab_type, distance_km):
    speed_kmph = {"mini": 30, "micro": 28, "sedan": 32, "shared": 25, "prime": 34}
    return round((distance_km / speed_kmph[cab_type]) * 60, 1) # minutes

def utility_function(cost, eta, cab_type, preferences):
    w_cost = preferences.get("cost_weight", 0.5)
    w_time = preferences.get("time_weight", 0.5)
    # lower cost and lower eta -> higher utility (inverted, normalized)
    return -(w_cost * cost + w_time * eta)

def ola_cab_agent(pickup_location, destination, distance_km, preferences):
    best_option = None
    best_utility = float("-inf")

    for cab_type in CAB_TYPES:
        cost = estimate_fare(cab_type, distance_km)
        eta = estimate_time(cab_type, distance_km)
        utility = utility_function(cost, eta, cab_type, preferences)

        if utility > best_utility:
            best_utility = utility
            best_option = {"cab_type": cab_type, "cost": cost, "eta_min": eta}

    return best_option

if __name__ == "__main__":
    preferences = {"cost_weight": 0.6, "time_weight": 0.4}
    booking = ola_cab_agent("Location A", "Location B", distance_km=12,
    preferences=preferences)
    print("Recommended cab:", booking["cab_type"])
    print(f"Estimated fare: Rs. {booking['cost']}")
    print(f"Estimated time: {booking['eta_min']} min")

```

Sample Output (12 km trip, cost-weighted preference):

Recommended cab: shared
 Estimated fare: Rs.72
 Estimated time: 28.8 min

With cost weighted more heavily than time (0.6 vs 0.4), the utility-based agent correctly recommends the cheapest option (shared). Increasing the time weight would shift the recommendation toward faster cabs such as sedan or prime.

Question 5 — Uniform Cost Search on the Delivery Network

Graph edges: S–A(1), S–G(12), A–B(3), A–C(1), B–D(3), C–D(1), C–G(2), D–G(3)

UCS Expansion Trace (priority queue by cumulative cost)

Step	Node Expanded	Frontier After Expansion (node: cost)
1	S (0)	A: 1, G: 12
2	A (1)	C: 2, B: 4, G: 12
3	C (2)	D: 3, G: 4 (updated, replaces 12), B: 4
4	D (3)	G: 4 (via D would be 6 — discarded, worse), B: 4
5	G (4)	Goal reached

All Candidate Paths (Verification)

- $S \rightarrow A \rightarrow C \rightarrow G = 1 + 1 + 2 = 4$ ☒ (minimum)
- $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G = 1 + 1 + 1 + 3 = 6$
- $S \rightarrow A \rightarrow B \rightarrow D \rightarrow G = 1 + 3 + 3 + 3 = 10$
- $S \rightarrow G = 12$

Result: The least-cost path is $S \rightarrow A \rightarrow C \rightarrow G$, with total cost 4 (fuel + time units).

Interpretation

UCS guarantees the optimal (lowest-cost) path because it always expands the frontier node with the smallest cumulative cost first, and it only terminates once the goal node is actually popped for expansion — not merely discovered. This is precisely why G, first found at cost 12 via the direct S–G edge, was correctly held back in the frontier until the cheaper path (cost 4, via A and C) was found and expanded.

Python Implementation (UCS using a priority queue)

```
import heapq
```

```
graph = {  
    "S": {"A": 1, "G": 12},  
    "A": {"B": 3, "C": 1},  
    "B": {"D": 3},  
    "C": {"D": 1, "G": 2},  
    "D": {"G": 3},  
    "G": {}  
}
```

```

def uniform_cost_search(graph, start, goal):
    frontier = [(0, start, [start])] # (cumulative_cost, node, path)
    visited = {}

    while frontier:
        cost, node, path = heapq.heappop(frontier)

        if node == goal:
            return cost, path

        if node in visited and visited[node] <= cost:
            continue
        visited[node] = cost

        for neighbor, edge_cost in graph[node].items():
            new_cost = cost + edge_cost
            if neighbor not in visited or new_cost < visited.get(neighbor, float("inf")):
                heapq.heappush(frontier, (new_cost, neighbor, path + [neighbor]))

    return None, None

if __name__ == "__main__":
    total_cost, path = uniform_cost_search(graph, "S", "G")
    print("Least-cost path:", " -> ".join(path))
    print("Total cost:", total_cost)

```

Sample Output:

Least-cost path: S -> A -> C -> G

Total cost: 4

The program's output matches the hand-worked UCS trace exactly, confirming the least-cost path and total cost are correct.